

AI CO-INVESTOR

Valura

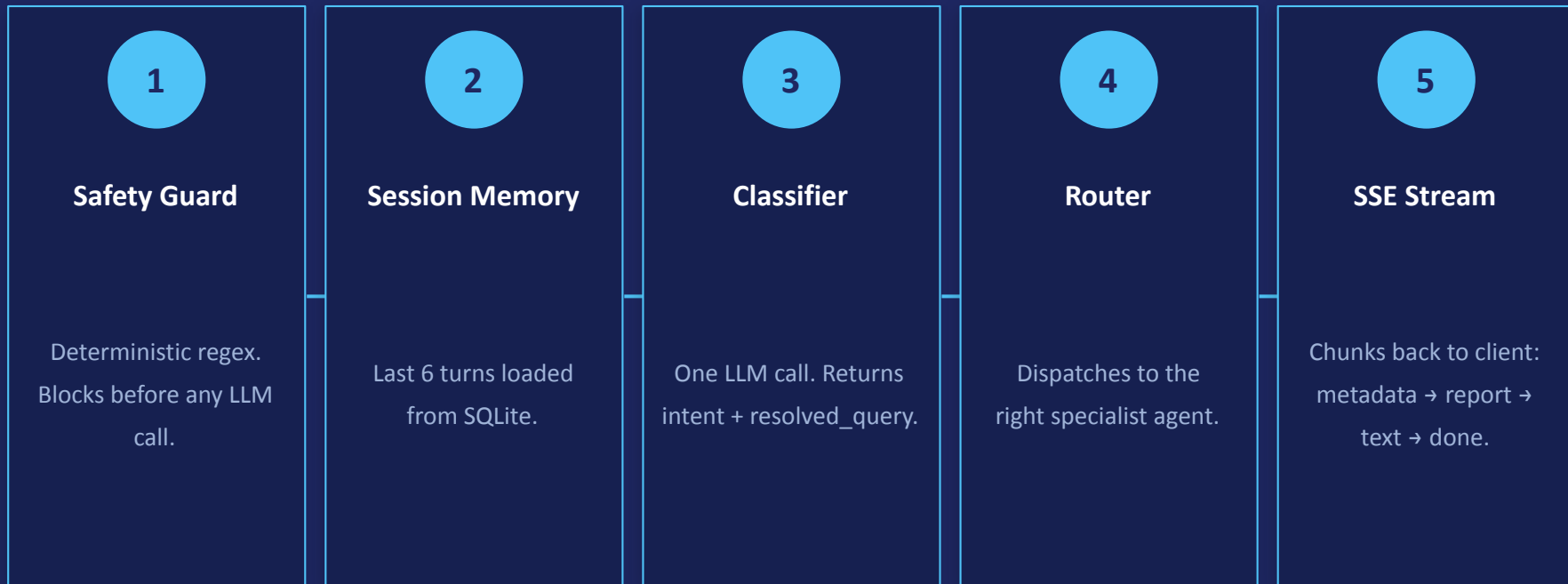
Build · Monitor · Grow · Protect

A specialist-agent microservice that gives every novice investor the guidance that used to require a human advisor.

FastAPI · SSE Streaming · 7 Specialist Agents · SQLite Session Memory

The Pipeline

Every query follows the same five-step flow — no exceptions.



Agent Ecosystem

One classifier. Seven specialists. The classifier always picks one — the pipeline never guesses.

LIVE

Portfolio Health

STUB

Risk Assessment

STUB

Market Research

STUB

Predictive Analysis

STUB

Investment Strategy

STUB

Support

STUB

Financial Calculator

All agents extend BaseAgent — NAME · LOG_COLOR · run() async generator

Portfolio Health Agent

The only fully-implemented agent in this build. Covers MONITOR + PROTECT.

How it works

- ① Check `portfolio.is_empty` → pivot to BUILD guidance if empty
- ② `_compute_concentration()` — top position %, top-3 %, flag
- ③ `_compute_performance()` — total return, annualised, best/worst
- ④ `BENCHMARK_MAP` lookup → yfinance fetch → alpha calculation
- ⑤ `_observations_stream()` — one LLM call, temperature 0.3, streams prose

Separation of concerns

Maths runs in pure Python — no LLM for arithmetic.
LLM called once, only to write plain-language observations.

Country-aware benchmarks

US → S&P 500 GB → FTSE 100

DE → DAX IN → BSE Sensex

Falls back gracefully if yfinance fails.

Safety Guard Architecture

The only layer that can block a request — runs before any LLM call.

Single source of truth

The classifier returns a `safety_verdict` too — but it's informational only. Emitted in the metadata event, never re-blocks. The guard is the gate.

Deterministic by design

Pure regex. No LLM involved. Runs in under 10ms regardless of model latency or hallucination. When it fires: blocked event → done event → exit. No classifier call, no agent call.

Educational framing tension

"Help me do a pump and dump" — blocked. "How does a pump and dump scheme work" — passes. Educational framing checked first. Accepted tradeoff for a fast first layer.

Session Memory + Follow-up Resolution

Multi-turn is where most agent systems fall apart. Valura resolves it in the classifier.

User: "Tell me about Microsoft" → **User:** "What about Apple?" → **resolved_query:** "Tell me about Apple stock"

6 turns injected

The last 6 turns of the session are loaded from SQLite and passed to the classifier as context before every call.

temperature: 0.0

The classifier must be deterministic.
The same query must always route to the same agent — no randomness.

SQLite, not in-memory

History must survive server restarts during development. In-memory would make testing multi-turn sessions fragile. SQLite costs nothing extra.

What I'd Do With Another Week

01

Market Research Agent

Portfolio health tells you where you are.

Market research tells you where opportunities are.

Together: MONITOR + GROW.

The stub is already routing correctly.

BaseAgent contract defined.

yfinance for data + one LLM call for narrative —
same pattern as portfolio health.

02

Split the Classifier

The classifier prompt does two jobs:
intent classification + follow-up resolution.

Split them:

- Embedding-based pre-classifier for the obvious 80% — cosine similarity, no LLM call
- LLM only for ambiguous queries

Cuts cost per query. Hits the 6s p95 target more consistently as the ecosystem grows.

The Spine Holds.

- All tests pass without an API key
- Safety guard · Classifier · Router · Portfolio Health Agent — production-ready
- 6 agents correctly wired as stubs — route, respond, never crash
- Next engineer: copy `portfolio_health.py` → implement `run()` → register one line in router